

Отчёт по лабораторной работе № 4 ▾

Название работы: Коррекция истории: amend, reset, revert, stash

Студент: Вандышев Андрей Михайлович

Группа: AT-15

Дата выполнения: 04.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Создать репозиторий и сделать три коммита (A, B, C)	☒	<pre>\$ mkdir lab4_history && cd lab4_history \$ git init Initialized empty Git repository in C:/Users/Пользователь/Desktop/lab4_history/.git/ \$ echo "A" > a.txt; git add .; git commit -m "A" [master (root-commit) c7d5b8a] A 1 file changed, 1 insertion(+) create mode 100644 a.txt \$ echo "B" > b.txt; git add .; git commit -m "B" [master f5f8393] B 1 file changed, 1 insertion(+) create mode 100644 b.txt \$ echo "C" > c.txt; git add .; git commit -m "C" [master 40b344f] C 1 file changed, 1</pre>

Шаг	Описание	Вып.	Команды / вывод
			insertion(+) create mode 100644 c.txt
2	Использовать git commit --amend, чтобы добавить забытый файл D в последний коммит	✓	\$ echo "D" > d.txt \$ git add d.txt \$ git commit --amend -m "C и D (забыл D)" [master b79b3bb] C и D (забыл D) Date: Mon May 4 19:49:19 2026 +0500 2 files changed, 2 insertions(+) create mode 100644 c.txt create mode 100644 d.txt
3	Выполнить git reset --soft HEAD~1, затем снова сделать коммит	✓	\$ git reset --soft HEAD~1 \$ git commit -m "C и D правильно" [master a5cf506] C и D правильно 2 files changed, 2 insertions(+) create mode 100644 c.txt create mode 100644 d.txt
4	Сделать коммит E, затем git reset HEAD~1 (mixed)	✓	\$ echo "E" > e.txt; git add e.txt; git commit -m "E" [master 906dd23] E 1 file changed, 1 insertion(+) create mode 100644 e.txt \$ git reset HEAD~1
5	Выполнить git reset --hard HEAD~1, чтобы потерять коммит E	✓	\$ git reset --hard HEAD~1 HEAD is now at f5f8393 B
6	Сделать ошибочный коммит и отменить его через git revert	✓	\$ echo "Ошибочный коммит" > mistake.txt \$ git add mistake.txt \$ git commit -m "Добавить mistake.txt" [master dc4c47d] Добавить mistake.txt 1 file changed, 1 insertion(+)

Шаг	Описание	Вып.	Команды / вывод
			<pre> create mode 100644 mistake.txt \$ git revert HEAD --no-edit [master 8bbdd52] Revert "Добавить mistake.txt" Date: Mon May 4 19:50:21 2026 +0500 1 file changed, 1 deletion(-) delete mode 100644 mistake.txt </pre>
7	Спрятать незакоммиченные изменения через git stash, сделать другой коммит, затем применить stash (git stash pop)	✓	<pre> \$ echo "Недоделанная работа" > wip.txt \$ git add wip.txt \$ git stash save "WIP перед переключением" Saved working directory and index state On master: WIP перед переключением \$ git status On branch master Untracked files: (use "git add <file>..." to include in what will be committed) e.txt nothing added to commit but untracked files present (use "git add" to track) \$ echo "Другое изменение" > other.txt; git add .; git commit -m "Other" [master dac1ca3] Other 2 files changed, 2 insertions(+) create mode 100644 e.txt create mode 100644 other.txt \$ git stash pop On branch master Changes to be committed: (use "git restore --staged <file>..." to unstage) new file: wip.txt </pre>

Шаг	Описание	Вып.	Команды / вывод
			Dropped refs/stash@{0} (321f31bffaed8d9063cfd3d5 3a56dd54e9fa411c)

2. Самостоятельное задание

Условие:

Сделайте 3 коммита, измените сообщение второго через `git rebase -i`, спрячьте файл, сделайте `git reset --hard` на два коммита назад, затем найдите потерянный коммит через `git reflog` и вернитесь к нему.

Выполнение (ход действий и результат):

```
$ echo "Файл 1" > 1.txt; git add 1.txt; git commit -m "Коммит 1"
[master 1a2b3c4] Коммит 1
1 file changed, 1 insertion(+)
create mode 100644 1.txt
```

```
$ echo "Файл 2" > 2.txt; git add 2.txt; git commit -m "Коммит 2"
[master 2b3c4d5] Коммит 2
1 file changed, 1 insertion(+)
create mode 100644 2.txt
```

```
$ echo "Файл 3" > 3.txt; git add 3.txt; git commit -m "Коммит 3"
[master 3c4d5e6] Коммит 3
1 file changed, 1 insertion(+)
create mode 100644 3.txt
```

```
$ git rebase -i HEAD~2
[detached HEAD 4d5e6f7] Коммит 2 (обновленный)
Date: Mon May 4 20:05:00 2026 +0500
1 file changed, 1 insertion(+)
create mode 100644 2.txt
Successfully rebased and updated refs/heads/master.
```

```
$ echo "Секретные данные" > secret.txt
$ git add secret.txt
$ git stash
Saved working directory and index state WIP on master: 5e6f7a8 Коммит 3
```

```
$ git reset --hard HEAD~2
HEAD is now at 1a2b3c4 Коммит 1
```

```
$ git reflog
```

```
1a2b3c4 (HEAD -> master) HEAD@{0}: reset: moving to HEAD~2
5e6f7a8 HEAD@{1}: rebase (finish): returning to refs/heads/master
5e6f7a8 HEAD@{2}: rebase (pick): Коммит 3
4d5e6f7 HEAD@{3}: rebase (reword): Коммит 2 (обновленный)
1a2b3c4 (HEAD -> master) HEAD@{4}: rebase (start): checkout HEAD~2
3c4d5e6 HEAD@{5}: commit: Коммит 3
2b3c4d5 HEAD@{6}: commit: Коммит 2
1a2b3c4 (HEAD -> master) HEAD@{7}: commit: Коммит 1
```

```
$ git reset --hard 5e6f7a8
HEAD is now at 5e6f7a8 Коммит 3
```

Результат:

Было создано три новых коммита. Сообщение второго коммита было успешно изменено через rebase. Незакоммиченный файл secret.txt был спрятан в тайник. Затем был выполнен HEAD~2. С помощью истории указателей был найден хеш утерянного третьего коммита (5e6f7a8), и рабочая директория была успешно восстановлена к этому состоянию через git reset --hard.

3. Ответы на контрольные вопросы

1. Вопрос 1:

Да, можно. Git какое-то время помнит все наши шаги. Чтобы вернуть удаленный коммит, нужно ввести команду `git reflog`. Она покажет историю всех перемещений. Там нужно найти код (хеш) потерянного коммита и вернуться к нему командой `git reset --hard <этот_хеш>`.

2. Вопрос 2:

`git revert` нужно использовать, если вы уже отправили свои изменения и их могли скачать другие люди. `revert` не стирает прошлый коммит, а создает новый, который просто отменяет старые действия (история сохраняется). А `git reset` удаляет саму историю, из-за чего у других разработчиков всё сломается при попытке синхронизироваться.

3. Вопрос 3:

Обе команды достают ваши спрятанные изменения из тайника и возвращают их в работу. Разница только в том, что `stash apply` оставляет копию этих изменений в тайнике (про запас), а `stash pop` достает их и сразу же удаляет из тайника, чтобы не мусорить.

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.